



ESCUELA DE INGENIERÍA DE FUENLABRADA

GRADO EN INGENIERÍA DE ROBÓTICA SOFTWARE

TRABAJO FIN DE GRADO

Desarrollo de un robot continuo accionado por tendones para inspección en entornos peligrosos.

Autora: Alba López del Águila

Tutor: Juan Sebastián Cely Gutiérrez

Curso académico 2025 / 2026

Resumen

Índice

Resumen	1
1. Introducción	6
1.1. Motivación	6
1.2. Problema abordado	6
1.3. Propuesta	6
1.4. Metodología del trabajo	6
2. Objetivos	6
2.1. Objetivos generales	6
2.2. Objetivos específicos	6
3. Estado del arte	6
3.1. Robots continuos	6
3.1.1. Categorías de los robots continuos	7
3.1.2. Partes de un robot continuo	7
3.1.3. Ventajas de un robot continuo frente a un robot rígido clásico	8
3.2. Tecnologías actuales en inspección	8
3.3. Herramientas de software	9
3.3.1. CoppeliaSim	9
3.3.2. ROS 2	9
3.3.3. Arduino	9
4. Diseño conceptual	9
4.1. Análisis de requerimientos	9
4.2. Morfología del robot	10
4.2.1. Tipo de accionamiento	10
4.2.2. Configuración de la columna central flexible	10
4.2.3. Estrategia de segmentos	11
4.2.4. Selección de tendones	12
4.3. Definición de la arquitectura propuesta	12
4.3.1. Descripción del concepto final	12
5. Modelado físico y matemático	13
5.1. Cinemática PCC	13
5.1.1. Cinemática directa	13
5.1.2. Transformación dependiente.	13
5.1.3. Transformación independiente: Método geométrico.	15
5.1.4. Cinemática inversa	17

5.2.	Modelado de fuerzas	19
5.3.	Definición del espacio de trabajo	20
6.	Simulación	22
6.1.	Entorno de simulación	22
6.2.	Implementación del algoritmo cinemático	22
6.3.	Validación del modelo	24
6.3.1.	Análisis cinemático	24
7.	Diseño detallado: hardware	29
7.1.	Diseño mecánico en CAD	29
7.1.1.	Diseño de segmentos	30
7.1.2.	Base del manipulador	30
7.1.3.	Polea de tendones	30
7.1.4.	Soporte	31
7.1.5.	Criterios de diseño	31
7.2.	Sistema de actuación	31
7.2.1.	Selección y dimensionamiento de actuadores	31
7.3.	Electrónica y control	33
7.3.1.	Microcontrolador	33
7.3.2.	Esquema eléctrico	33
7.4.	Fabricación	33
8.	Diseño detallado: arquitectura software	33
8.1.	Integración en ROS 2	33
8.1.1.	Creación del paquete URDF/Xacro del robot	33
8.1.2.	Integración de ros2_control	34
8.1.3.	Plugin hardware: CableRobotHardware	35
8.1.4.	Definición de nodos	35
8.1.5.	Cinemática directa: modelo PCC	36
8.1.6.	Controlador de posición en espacio de cables	37
8.1.7.	Grafo de la arquitectura	38
8.1.8.	Modos de operación y comando del robot	38
8.1.9.	Configuración de RViz para visualización	40
8.1.10.	Interfaz gráfica de usuario	40
8.2.	Firmware: Arduino	40
8.2.1.	Control a bajo nivel	40
8.2.2.	Librerías Arduino para operación autónoma sin ROS	41
9.	Resultados y pruebas experimentales	43

10. Conclusiones	43
Bibliografía	44
A. Anexos	45

Índice de figuras

1.	Anatomía de un robot continuo.	8
2.	La fuerza en el tendón provoca flexión en la dirección del orificio guía	11
3.	Planta de la sección del robot.	14
4.	Dibujo esquemático para cálculos de cinemática directa—PCC	15
5.	Deducción del movimiento de traslación para el punto final.	16
6.	Validación de la cinemática directa. La figura compara la posición real y teórica del efector final, y muestra la evolución del error cartesiano.	25
7.	Validación de la cinemática directa. Muestra la evolución del error cartesiano. .	26
8.	Validación de la cinemática inversa. La figura muestra la comparación entre la posición objetivo.	28
9.	Validación de la cinemática inversa. La figura muestra las longitudes efectivas de los cables.	29
10.	Grafo de la arquitectura software en ROS2	38

1. Introducción

1.1. Motivación

1.2. Problema abordado

1.3. Propuesta

1.4. Metodología del trabajo

2. Objetivos

2.1. Objetivos generales

2.2. Objetivos específicos

3. Estado del arte

Comúnmente los manipuladores utilizados tienen eslabones rígidos, conectados por un número de grados de libertad. Este número de grados de libertad es suficiente para cierto tipo de tareas donde cada articulación tiene un movimiento definido. Esto supone ciertas limitaciones, principalmente relacionadas con tareas en entornos poco accesibles o limitados y la manipulación de objetos usando las partes del manipulador y no el extremo final. En la naturaleza, estas limitaciones están resueltas con estructuras complejas de algunas especies como la trompa de un elefante o el tentáculo de un pulpo. Los robots continuos basan su estructura, movimiento y funcionamiento en la naturaleza.

3.1. Robots continuos

Los robots continuos son sistemas robóticos que se caracterizan por su flexibilidad y composición. En lugar de estar formados por eslabones rígidos, como ocurre en los robots rígidos convencionales, los robots continuos están compuestos por cuerpos flexibles que pueden curvarse y deformarse de manera progresiva gracias a la elasticidad de su estructura.

Además, se trata de sistemas subactuados, ya que un solo actuador puede generar movimiento a lo largo de varios o infinitos ejes. Esto les permite deformarse de manera continua y moverse por entornos peligrosos o simplemente imposibles para un robot rígido convencional [5].

Sus aplicaciones más comunes actualmente se encuentran en la cirugía mínimamente invasiva. El robot asiste al cirujano, proporcionando estabilidad, precisión y comodidad. Además, proporciona al paciente una mejor recuperación debido a la mínima invasión.

Por otro lado, la inspección es otra de las aplicaciones de los robots continuos. Gracias a su

versatilidad de movimientos les permite acceder a espacios donde es inaccesible para el ser humano o no es posible durante grandes periodos de tiempo.

3.1.1. Categorías de los robots continuos

Existen distintos tipos de robots continuos y diferentes categorías donde clasificarlos, sin embargo, cabe destacar:

- Robot continuo actuado por tendones.

Estos son mecanismos accionados por tendones. A grandes rasgos, su estructura consiste en una columna vertebral central que puede estar hecha de materiales altamente flexibles y unos segmentos por donde pasan los cables. La actuación del manipulador surge de la variación de longitud de los tendones, modificando la curvatura del robot. [4]

- Robot continuo actuado por neumático o hidráulico.

Estos son similares a los mencionados anteriormente, sin embargo, cambia su estructura de actuación. Mediante actuadores neumáticos que utilizan aire para lograr movimiento en ciertas partes del robot o mediante actuadores hidráulicos que utilizan agua u otros líquidos como aceites. Estos robots continuos tienen cámaras internas que se llenan o se vacían, la sincronización de varias cámaras deriva en movimientos como doblarse, torcerse o extenderse en una dirección. Gracias a sus materiales blandos, les permiten interactuar con el entorno de manera segura. [4]

- Robot continuo actuado por magnetismo.

El movimiento de estos robots se consigue mediante la activación de imanes en ciertas zonas del manipulador. Aunque este sistema tenga mucho potencial, presenta una alta limitación en la debilidad de la fuerza magnética ya que no se podría usar para tareas que requieran una fuerza mayor. [4]

3.1.2. Partes de un robot continuo

Como este trabajo se ha centrado en los robots continuos actuados por tendones se va a explicar sus partes pero, como se ha mencionado anteriormente, para los otros tipos de robots continuos sus partes no varían en su mayoría.

Un robot actuado por tendones está formado principalmente por una columna vertebral, tendones, secciones donde puede haber uno o varios segmentos o discos, y el sistema de actuación. La columna vertebral, como elemento estructural principal, proporciona la figura y mantiene el peso del sistema junto con los tendones. Los tendones, conectados a los actuadores, son los encargados del movimiento continuo del robot. Los discos o segmentos proporcionan la forma exterior del robot, además cuentan con unos orificios colocados de manera estratégica que sirven como guía para los tendones.

A partir de estos elementos estructurales se desarrolla posteriormente el modelo cinemático del manipulador.

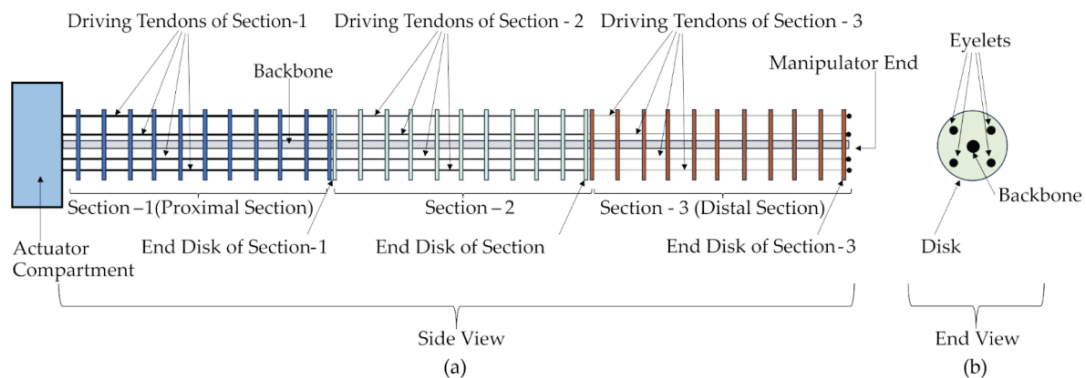


Figura 1: Anatomía de un robot continuo.

3.1.3. Ventajas de un robot continuo frente a un robot rígido clásico

La principal ventaja de los robots continuos es su flexibilidad. Gracias a su número de grados de libertad estos robots pueden alcanzar poses que ayudan a la interacción con el entorno. Además, a diferencia de los robots rígidos que precisan de un suplemento como una pinza, estos robots pueden enrollar su cuerpo para agarrar cualquier tipo de objeto sin dañarlo. Aunque estos objetos deben ser livianos ya que estos robots continuos no alcanzan a soportar objetos muy pesados.

3.2. Tecnologías actuales en inspección

Los robots en inspección desempeñan un papel fundamental en el aumento de la seguridad y la eficiencia operativa. Estos robots realizan tareas en ámbitos como industrial, construcción, salud o intervención en desastres, entre otros. Además, existe una gran variedad de robots que llevan a cabo las tareas de inspección como robots aéreos o drones, generalmente utilizados para inspección de grandes superficies por el aire. Estos sistemas llevan incorporados sensores como cámaras que nos proporcionan información detallada del entorno.

Por otro lado, los robots terrestres se utilizan para inspeccionar superficies a nivel del suelo. Se emplean para la manufactura, logística y control de cintas mecánicas, control de construcciones... Están diseñados para la navegación en entornos no controlados, lo que les permite tener una gran versatilidad y efectividad.

Los robots continuos están comenzando a adquirir un papel importante en tareas de inspección debido a su capacidad para adaptarse a entornos complejos, estrechos o de difícil acceso. A diferencia de los robots rígidos convencionales, su estructura flexible les permite curvarse y rodear obstáculos, facilitando la exploración de zonas donde otros sistemas no podrían operar.

Además, muchos de estos robots presentan una arquitectura en la que los actuadores se encuentran

separados del manipulador, normalmente situados en una base fija o protegida. Esto permite que la parte móvil sea más ligera, compacta y segura, algo especialmente útil en entornos hostiles como medios acuáticos, tuberías, instalaciones industriales o aplicaciones espaciales. Gracias a esta configuración, los robots continuos pueden transportar sensores o cámaras hasta zonas de difícil acceso, mejorando la capacidad de inspección sin comprometer la integridad del sistema.

3.3. Herramientas de software

3.3.1. CoppeliaSim

CoppeliaSim es un entorno de simulación robótica basado en una arquitectura de control distribuido, donde cada objeto de la escena puede ser controlado mediante scripts en Lua o Python, API remotas, complementos o plugins [2]. En este proyecto se ha utilizado principalmente para la creación rápida de prototipos y la verificación inicial del comportamiento del manipulador en un entorno simulado. La elección de CoppeliaSim frente otros simuladores radica en las relaciones de dependencia las cuales permiten vincular el movimiento de un joint al de otro joint, esencial para la simulación de flexión del manipulador.

3.3.2. ROS 2

ROS 2 se ha utilizado en este proyecto como herramienta principal para la representación y visualización del robot en simulación. A partir de los modelos URDF y Xacro, se ha definido la estructura del manipulador, incluyendo sus eslabones, geometría y relaciones cinemáticas. Además, se ha empleado RViz para desarrollar una interfaz gráfica que permite visualizar el comportamiento del robot y relacionar la longitud de los cables con la flexión del manipulador continuo.

3.3.3. Arduino

Arduino se ha utilizado para el desarrollo del modelo físico del robot. Su uso permite programar de forma sencilla el control de los actuadores y trasladar al prototipo real los conceptos previamente validados en simulación. De esta manera, Arduino actúa como una plataforma práctica para implementar el movimiento del manipulador y comprobar experimentalmente su funcionamiento.

4. Diseño conceptual

4.1. Análisis de requerimientos

Para llevar a cabo un diseño conceptual se tiene que analizar los requisitos asociados a una tarea de inspección en entornos peligrosos. En este tipo de aplicaciones el manipulador debe ser capaz

de adaptarse a geometrías estrechas alcanzando zonas inaccesibles para un manipulador rígido convencional. Además de adaptarse a cualquier tipo de entorno.

Para ello, se establece como requisitos principales:

- Diseño compacto.
- Diseño ligero.
- Diseño modular.
- Diseño de bajo coste.
- Diseño con alta movilidad en varias direcciones.
- Diseño de fácil fabricación y reproducible.

4.2. Morfología del robot

4.2.1. Tipo de accionamiento

Entre las distintas alternativas de accionamiento descritas en el estado del arte, para este proyecto se selecciona el accionamiento por tendones. Esta opción resulta adecuada para un manipulador de inspección porque permite situar los actuadores fuera del cuerpo flexible del robot, reduciendo la masa móvil y simplificando la estructura del módulo continuo.

Frente a soluciones neumáticas, hidráulicas o magnéticas, el accionamiento por tendones presenta una implementación mecánica más sencilla y económica, ya que puede realizarse mediante cables, poleas y servomotores. Además, esta configuración es compatible con fabricación mediante impresión 3D y facilita el mantenimiento del sistema, puesto que los tendones pueden sustituirse sin modificar el resto de la estructura.

4.2.2. Configuración de la columna central flexible

La columna central flexible constituye el elemento estructural principal del manipulador continuo. Su función es proporcionar continuidad mecánica entre los segmentos, soportar el peso del conjunto y aportar la elasticidad necesaria para recuperar la posición neutra cuando se reduce la tensión de los tendones.

Existen diferentes configuraciones posibles para este elemento, como muelles, varillas flexibles, tubos de fibra de carbono o materiales blandos como siliconas. La elección de la columna central flexible condiciona directamente la rigidez del robot, su capacidad de curvatura, la repetibilidad del movimiento y la fuerza necesaria para accionar los tendones.

En este diseño se plantea una columna central flexible de longitud aproximadamente constante. Esta solución simplifica el modelo cinemático, ya que permite aproximar cada tramo del robot

mediante curvatura constante por tramos, y facilita la fabricación del prototipo. La columna central actúa como eje de referencia del manipulador, mientras que los segmentos separadores mantienen la posición relativa de los tendones respecto a dicho eje.

4.2.3. Estrategia de segmentos

Los segmentos son una estructura de soporte fundamental para los robots accionados por tendones. Su función es definir la forma periférica del módulo continuo, guiar los tendones y contribuir a determinar el número de grados de libertad. El número de segmentos incorporados al diseño influye en la forma continua y suave de los tendones.

Los segmentos pueden clasificarse en tres tipos: discos, estructuras con incisiones y estructuras tipo vértebras. Además, pueden categorizarse según su libertad de movimiento respecto a la columna vertebral. Se pueden fijar de la siguiente forma: tipo 0, no es posible ningún movimiento; tipo 1, solo es posible la traslación; tipo 2, donde es posible la rotación y por último tipo 3 donde es posible tanto la traslación como la rotación.

Además, los segmentos sirven de guía para los tendones, los segmentos tienen orificios guía por los que pasa cada tendón, por lo que se tienen tantos orificios como número de tendones. La disposición de los orificios o enrutamiento del tendón es crucial para el movimiento del manipulador, ya que, cuando se acciona un tendón se aplica una fuerza que deforma el esqueleto flexible en la dirección del orificio. [6]

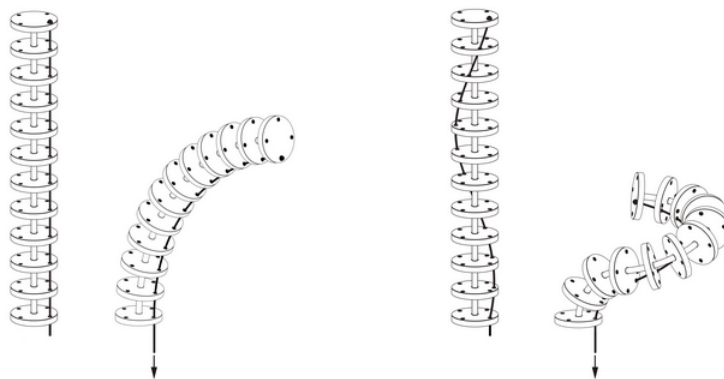


Figura 2: La fuerza en el tendón provoca flexión en la dirección del orificio guía

Algunos de los enrutamientos de tendones más comúnmente utilizados son:

Diseño A: este diseño emplea una ruta del tendón parcialmente restringida, en la que la trayectoria del tendón se encuentra directamente dentro de un subsegmento, es decir, entre dos discos espaciadores individuales, formando una serie de segmentos de línea a lo largo de la columna vertebral. [6]

Diseño B: este diseño emplea un camino tendinoso totalmente restringido, en el que se supone que los tendones siguen una curva continua y paralela a la columna vertebral. [6]

4.2.4. Selección de tendones

Al seleccionar los tendones, se debe tener en cuenta la fuerza máxima que el manipulador podría inducir sobre un tendón. Además, los tendones deben presentar una baja elongación bajo las fuerzas aplicadas con el fin de evitar desviaciones no deseadas en la forma del manipulador.

Para lograrlo, es recomendable elegir tendones con alta resistencia a la tracción, capaces de soportar fuerzas significativas, y con un módulo de Young elevado, para garantizar una extensión mínima. Es necesario estimar la fuerza máxima que el módulo continuo diseñado deberá soportar y, a partir de ello, seleccionar el tendón adecuado.

Para lograr estas características los tendones de los manipuladores suelen fabricarse con hilos trenzados, nailon, hilo de pesca o cables de tungsteno, entre otros.

Además de sus propiedades mecánicas, la disposición y el número de tendones influyen en la capacidad de actuación del manipulador. En robots continuos accionados por tendones, una distribución de al menos tres tendones alrededor de la columna vertebral permite generar flexión espacial, asociada normalmente a dos grados de libertad de curvatura. No obstante, el aumento del número de tendones no implica necesariamente un incremento directo del número de grados de libertad independientes, ya que varias tensiones pueden estar acopladas y contribuir conjuntamente a los mismos modos de deformación. [3]

4.3. Definición de la arquitectura propuesta

4.3.1. Descripción del concepto final

La arquitectura propuesta consiste en un manipulador continuo accionado por tres tendones distribuidos de forma equidistante alrededor de una columna central flexible. Esta columna actúa como elemento estructural principal, proporcionando continuidad mecánica, soporte y capacidad de recuperación elástica cuando disminuye la tensión aplicada en los tendones.

El diseño se orienta a tareas de inspección en entornos peligrosos, estrechos o de difícil acceso, por lo que se prioriza una solución compacta, ligera, modular, de bajo coste y fácil de fabricar. Para ello, se selecciona un accionamiento por tendones, ya que permite situar los actuadores fuera del cuerpo flexible del robot, reduciendo la masa móvil del manipulador y simplificando tanto la construcción como el mantenimiento del sistema.

Los tendones atraviesan una serie de segmentos separadores de tipo 0, cuya función es mantener fija la posición relativa de los cables respecto al eje central y guiar correctamente su trayectoria mediante orificios guía. La distribución de tres tendones permite generar flexión espacial del

manipulador, asociada a dos grados de libertad de curvatura. Al variar la longitud efectiva de cada tendón, se produce una deformación controlada de la columna central flexible, permitiendo orientar el extremo del robot en diferentes direcciones.

El concepto final combina, por tanto, una estructura flexible central, segmentos separadores fabricables mediante impresión 3D y tendones de baja elongación y alta resistencia a la tracción. Esta configuración proporciona una solución sencilla, reproducible y compatible con el modelado cinemático mediante curvatura constante por tramos, adecuada para el desarrollo de un prototipo funcional de robot continuo destinado a inspección.

5. Modelado físico y matemático

5.1. Cinemática PCC

El método de curvatura constante por tramos (PCC) se basa en aproximar la forma del robot como una sucesión finita de arcos de curvatura constante. Es decir, en lugar de modelar cada punto del cuerpo continuo de forma independiente, el robot se divide en varias secciones, y cada una de ellas se representa mediante un arco circular.

Las condiciones generales para asumir esta hipótesis son que la fuerza de la gravedad es despreciable y que el segmento por donde pasan los tendones está fijo a la columna central, permitiendo que el robot se curve y produciendo una fricción despreciable. Los arcos de curvatura o secciones del manipulador normalmente corresponden al espacio que hay entre dos segmentos o discos.[1]

5.1.1. Cinemática directa

Una vez aceptada la hipótesis debemos definir los parámetros de esta. Cada sección del manipulador se modela mediante tres valores: curvatura k , ángulo acimutal ϕ y longitud del arco l .

La cinemática directa puede entenderse como un proceso de dos transformaciones. Primero, se obtiene la relación entre las longitudes de los tendones y los parámetros de curvatura de la sección. Esta parte depende de la geometría concreta del robot, por ejemplo del número de tendones, su distribución alrededor del eje central y la distancia radial respecto a la columna central flexible. Después, una vez conocidos k , ϕ y l , se calcula la transformación homogénea que describe la pose del extremo de la sección respecto a su base. [1]

5.1.2. Transformación dependiente.

En un robot continuo accionado por tendones, la curvatura de una sección se produce por la diferencia de longitud entre los cables. Si todos los tendones mantienen la misma longitud, la

sección tiende a permanecer recta. Por el contrario, si uno de los tendones se acorta respecto a los demás, la sección se curva hacia el lado de dicho tendón. Por tanto, esta transformación permite obtener los parámetros PCC de la sección, es decir, la curvatura κ , el ángulo azimutal ϕ y la longitud de arco l , a partir de las longitudes de los tendones. [1]

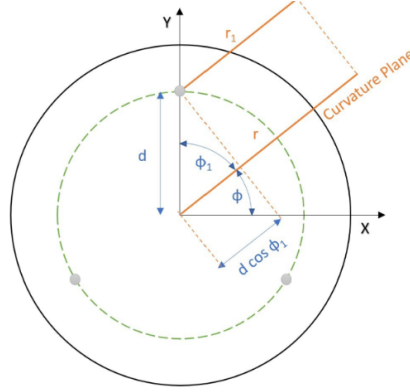


Figura 3: Planta de la sección del robot.

En la Figura 3, observamos la proyección en el plano de uno de los discos de la sección, vemos el plano de curvatura así como el plano paralelo que contiene la curva que sigue el cable 1 (ambos en naranja). Suponiendo un manipulador de tres tendones activos, distribuidos de manera homogénea y equidistante del centro de la sección con un radio d , cada tendón i queda definido por un ángulo ϕ_i dentro de la sección. Bajo esta configuración, el radio descrito por cada tendón puede expresarse como:

$$r_i = r - d \cos(\phi_i) \quad (1)$$

donde r es el radio de curvatura de la columna central flexible y r_i es el radio de curvatura asociado al tendón i . Esta expresión muestra que cada tendón recorre un arco de radio ligeramente diferente dependiendo de su posición respecto al plano de curvatura.

A partir de esta relación geométrica, la longitud de la columna central flexible de la sección y la longitud de cada tendón pueden escribirse como:

$$l_c = 2pr \sin\left(\frac{\theta}{2p}\right) \quad (2)$$

$$l_i = 2pr_i \sin\left(\frac{\theta}{2p}\right) \quad (3)$$

donde p representa el número de discos pasivos de la sección, θ es el ángulo total de curvatura y l_i es la longitud del tendón i . Sustituyendo la expresión de r_i , se obtiene una relación directa entre la longitud del tendón y la geometría de la sección:

$$l_c = l_i + 2pd \sin\left(\frac{\theta}{2p}\right) \cos(\phi_i) \quad (4)$$

Esta ecuación permite ver que la longitud de cada tendón depende de su posición angular dentro de la sección. Por ello, las diferencias entre las longitudes de los tendones permiten determinar tanto la magnitud de la curvatura como la dirección hacia la que se curva la sección.

En el caso de tres tendones distribuidos de forma homogénea, la curvatura puede obtenerse a partir de sus longitudes mediante:

$$\kappa = \frac{2\sqrt{l_1^2 + l_2^2 + l_3^2 - l_1l_2 - l_1l_3 - l_2l_3}}{d(l_1 + l_2 + l_3)} \quad (5)$$

De esta forma, la transformación dependiente proporciona el vínculo entre el accionamiento real del robot y el modelo PCC. Una vez obtenidos los parámetros κ , ϕ y l , puede aplicarse la transformación independiente para calcular la matriz homogénea de la sección [1].

5.1.3. Transformación independiente: Método geométrico.

Se puede definir un arco 3D en el espacio, con el origen O del sistema de referencia ubicado en el centro de su base inferior (véase la Figura 2). El eje z se define como ortogonal a dicha sección y tangente a la curva del arco. Sería deseable que el eje x apuntara al centro del arco, pero esto no siempre es posible. Por lo tanto, para no perder generalidad, el eje x puede definirse arbitrariamente, siempre que sea ortogonal al eje z. El eje y se obtiene mediante el producto vectorial, manteniendo la ortogonalidad.

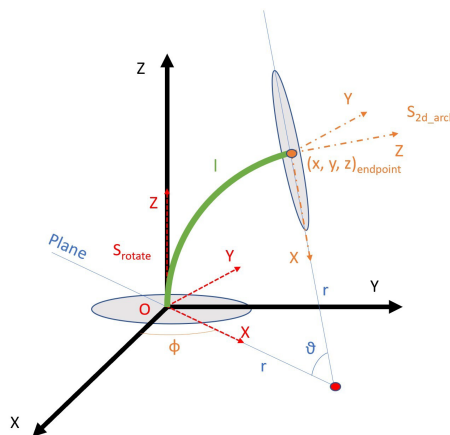


Figura 4: Dibujo esquemático para cálculos de cinemática directa—PCC

El arco usa los parámetros de curvatura de la sección, pero el problema está en encontrar la relación entre los parámetros de curvatura y el punto final del arco (x, y, z). Puede lograrse

combinando dos movimientos: primero, una rotación de ϕ respecto al eje Z; y segundo, una combinación de una traslación que sitúa el extremo en la sección final y una rotación que logra que el eje Z sea tangente a la sección y que el eje X apunte al centro de la rotación.

$${}^{k-1}A_k = T_{\text{rotate}}(\phi) \cdot T_{2d_arch}(\kappa, l) \quad (6)$$

Geoméricamente, una sección PCC puede interpretarse como un arco contenido en un plano. Para representar todo el arco y no solo un extremo se introduce una nueva variable s , donde $s \in [0, l]$.

El radio del arco viene dado por:

$$r = \frac{1}{\kappa} \quad (7)$$

mientras que el ángulo barrido por el arco se obtiene mediante:

$$\theta = \kappa s \quad (8)$$

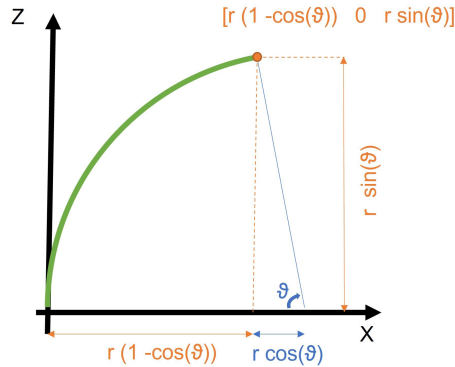


Figura 5: Deducción del movimiento de traslación para el punto final.

Combinando estos dos parámetros y observando la geometría de la Figura 3:

$$\mathbf{t} = \begin{bmatrix} r(1 - \cos \theta) & 0 & r \sin \theta \end{bmatrix} \quad (9)$$

$$\theta = \begin{bmatrix} \frac{1-\cos(\kappa s)}{\kappa} & 0 & \frac{\sin(\kappa s)}{\kappa} \end{bmatrix} \quad (10)$$

Combinamos la traslación y la rotación:

$$T_{2d_arch} = \begin{bmatrix} \cos(\kappa s) & 0 & \sin(\kappa s) & \frac{1 - \cos(\kappa s)}{\kappa} \\ 0 & 1 & 0 & 0 \\ -\sin(\kappa s) & 0 & \cos(\kappa s) & \frac{\sin(\kappa s)}{\kappa} \\ 0 & 0 & 0 & 1 \end{bmatrix} \quad (11)$$

Las ecuaciones cinemáticas se obtienen multiplicando la matriz de rotación de ϕ respecto al eje Z y la matriz de la Figura 6, obteniendo la matriz general:

$${}^{k-1}A_k(l) = \begin{bmatrix} \cos \phi \cos(\kappa l) & -\sin \phi & \cos \phi \sin(\kappa l) & \frac{\cos \phi (1 - \cos(\kappa l))}{\kappa} \\ \sin \phi \cos(\kappa l) & \cos \phi & \sin \phi \sin(\kappa l) & \frac{\sin \phi (1 - \cos(\kappa l))}{\kappa} \\ -\sin(\kappa l) & 0 & \cos(\kappa l) & \frac{\sin(\kappa l)}{\kappa} \\ 0 & 0 & 0 & 1 \end{bmatrix} \quad (12)$$

5.1.4. Cinemática inversa

En un manipulador continuo, la cinemática inversa busca la relación entre la longitud de los cables o tendones y la posición final. Como en el modelo anterior necesitamos dos transformaciones, una de ellas es la definición geométrica de los parámetros de curvatura y después se necesita una transformación dependiente del manipulador, equivalente a la de la cinemática directa. [1]

En primer lugar, es necesario obtener los parámetros de curvatura de cada sección dado un cierto punto final. La orientación se fija cuando se dan tres parámetros, ya que los parámetros de curvatura implican que el manipulador tenga tres grados de libertad por sección.

Para comenzar, el ángulo acimutal se obtiene fácilmente mediante:

$$\phi = \arctan\left(\frac{Y}{X}\right) \quad (13)$$

La curvatura k se obtiene buscando el radio analizando el arco del robot en el plano vertical que lo contiene, ya que $k = 1/R$. Para obtener el radio lo igualamos a la distancia desde el centro del arco hasta el punto final.

$$(\sqrt{x^2 + y^2} - R)^2 + z^2 = R^2 \quad (14)$$

Despejamos R para obtener k :

$$k = \frac{2\sqrt{x^2 + y^2}}{x^2 + y^2 + z^2} \quad (15)$$

Por último, nos falta el ángulo θ :

$$\theta = \begin{cases} \cos^{-1} \left(1 - \kappa \sqrt{x^2 + y^2} \right), & z > 0 \\ 2\pi - \cos^{-1} \left(1 - \kappa \sqrt{x^2 + y^2} \right), & z \leq 0 \end{cases}$$

Una vez obtenidos los parámetros de curvatura, debemos obtener la longitud de los cables necesaria para lograr dicha configuración. Esta transformación es dependiente del manipulador, por lo que esta explicación es para un modelo cinemático con un manipulador de tres cables. Para ello combinamos las ecuaciones (3) y (4) que definen la longitud de los cables:

$$l_i = 2p \sin \left(\frac{\theta}{2p} \right) \left(\frac{1}{\kappa} - d \sin(\phi + \phi_i) \right) \quad (16)$$

donde p representa el número de segmentos pasivos (no conectados a un cable). En el caso límite en el que el cable se aproxima como una curva continua, o cuando el número de segmentos es suficientemente grande, se cumple:

$$2p \sin \left(\frac{\theta}{2p} \right) \approx \theta \quad (17)$$

por lo que la expresión anterior puede escribirse como:

$$l_i \approx \frac{\theta}{\kappa} - d\theta \sin(\phi + \phi_i) \quad (18)$$

Dado que la longitud del eje central del segmento es $l_c = \theta/\kappa$, la variación de longitud del cable respecto a dicho eje central queda:

$$\Delta l_i \approx -d\theta \sin(\phi + \phi_i) \quad (19)$$

En la implementación se emplea la magnitud de esta variación como:

$$c_i = d\theta \sin(\phi + \phi_i) \quad (20)$$

de modo que la longitud efectiva del cable puede expresarse como:

$$l_i \approx l_c - c_i \quad (21)$$

Por tanto, c_i no representa la longitud absoluta del cable, sino el acortamiento o alargamiento

relativo necesario respecto a la configuración neutra.

5.2. Modelado de fuerzas

El modelado de fuerzas del manipulador se plantea de forma simplificada, con el objetivo de identificar los esfuerzos principales que aparecen durante el accionamiento por tendones y justificar las decisiones de diseño mecánico. No se pretende realizar un análisis estructural completo del sistema, sino establecer una relación básica entre la tensión de los cables, la flexión del manipulador y las cargas transmitidas a la estructura.

En un robot continuo accionado por tendones, el movimiento se genera mediante diferencias de tensión entre los cables. Cuando un tendón se acorta, aparece una fuerza de tracción que produce un momento respecto al eje central del manipulador. Este momento provoca la curvatura de la columna central en la dirección del tendón accionado. Si los tres tendones mantienen tensiones similares, los momentos se compensan y el robot tiende a permanecer en una configuración aproximadamente recta. [6]

Considerando un tendón situado a una distancia radial d respecto al eje central del manipulador, la tensión T_i aplicada sobre dicho tendón genera un momento aproximado:

$$M_i = dT_i \quad (22)$$

donde M_i es el momento producido por el tendón i , d es la distancia entre el tendón y el eje central, y T_i es la tensión del tendón. Esta expresión muestra que, para una misma tensión, aumentar la distancia radial del tendón incrementa la capacidad de flexión del manipulador. Sin embargo, también aumenta el diámetro exterior del robot, por lo que existe un compromiso entre movilidad y compacidad.

Desde el punto de vista de la columna central, la flexión del robot puede relacionarse con la rigidez del material mediante la expresión clásica de flexión:

$$M = EI\kappa \quad (23)$$

donde E es el módulo de Young del material, I es el segundo momento de área de la sección transversal y κ es la curvatura. Esta relación indica que, cuanto mayor sea la rigidez de la columna central, mayor será el momento necesario para alcanzar una determinada curvatura. Por el contrario, una columna central demasiado flexible permitiría una gran deformación, pero reduciría la estabilidad y la repetibilidad del movimiento. [6]

El sistema de actuación también introduce una relación directa entre el par del motor y la tensión generada en el tendón. Si la polea tiene un radio r_p , el par necesario para aplicar una tensión T_i

puede aproximarse como:

$$\tau_i = T_i r_p \quad (24)$$

donde τ_i es el par aplicado por el motor y r_p es el radio efectivo de la polea. Esta relación es importante para la selección de los actuadores, ya que una polea de mayor radio permite recoger más cable por cada giro, pero requiere un par mayor para generar la misma tensión. Por el contrario, una polea de menor radio reduce el par necesario, aunque disminuye la variación de longitud obtenida por vuelta. [6]

Las fuerzas generadas por los tendones no solo actúan sobre el manipulador, sino que también se transmiten al sistema de poleas, rodamientos y soportes. Por este motivo, las piezas de soporte deben mantener la alineación del cable y resistir las cargas producidas durante el accionamiento. En particular, el soporte del rodamiento debe impedir desplazamientos laterales de la polea y absorber las fuerzas derivadas de la tensión del tendón, transmitiéndolas a la base del sistema.

En este trabajo, el modelo de fuerzas se utiliza principalmente como criterio de diseño. A partir de él se justifica la necesidad de emplear tendones de baja elongación, una columna central suficientemente flexible pero estable, poleas correctamente alineadas y soportes capaces de contrarrestar las cargas transmitidas por los cables.

5.3. Definición del espacio de trabajo

El espacio de trabajo del manipulador se define como el conjunto de posiciones que puede alcanzar el extremo final del robot para todas las configuraciones admisibles de sus tendones. En un robot continuo, este espacio no depende únicamente de los límites de articulaciones discretas, como ocurre en un robot rígido convencional, sino de la curvatura máxima alcanzable, de la longitud del manipulador, de la disposición de los tendones y de las limitaciones mecánicas del sistema.

Bajo la hipótesis de curvatura constante por tramos, una sección del manipulador de longitud l puede describirse mediante los parámetros κ , ϕ y l . La posición del extremo de una sección con curvatura constante viene dada por:

$$x = \frac{\cos \phi (1 - \cos(\kappa l))}{\kappa} \quad (25)$$

$$y = \frac{\sin \phi (1 - \cos(\kappa l))}{\kappa} \quad (26)$$

$$z = \frac{\sin(\kappa l)}{\kappa} \quad (27)$$

Estas ecuaciones, obtenidas de la matriz (12), muestran que el extremo del manipulador describe una superficie de revolución alrededor del eje vertical cuando se varía el ángulo azimutal ϕ . Para una curvatura fija, cambiar ϕ modifica la dirección de flexión, mientras que cambiar κ modifica la magnitud de la curvatura y, por tanto, la distancia del extremo respecto al eje central.

Cuando la curvatura tiende a cero, el manipulador se encuentra en su configuración recta. En ese caso, las expresiones anteriores presentan una indeterminación matemática, pero físicamente el extremo se sitúa sobre el eje vertical:

$$\lim_{\kappa \rightarrow 0} x = 0, \quad \lim_{\kappa \rightarrow 0} y = 0, \quad \lim_{\kappa \rightarrow 0} z = l \quad (28)$$

Por tanto, la posición recta representa el alcance máximo en altura del manipulador.

El límite del espacio de trabajo viene determinado por la curvatura máxima admisible κ_{max} . Esta curvatura máxima no depende solo del modelo matemático, sino también de restricciones físicas como la elongación máxima de los tendones, el recorrido disponible en las poleas, el par máximo de los actuadores, la rigidez de la columna central flexible, posibles colisiones entre segmentos y el radio mínimo de curvatura que puede soportar la estructura sin dañarse.

A partir de la relación entre curvatura y ángulo total de flexión:

$$\theta = \kappa l \quad (29)$$

puede definirse el intervalo de movimiento admisible como:

$$0 \leq \theta \leq \theta_{max} \quad (30)$$

donde θ_{max} representa la flexión máxima permitida por el diseño. Para cada valor de θ , el ángulo ϕ puede variar entre 0 y 2π , generando diferentes direcciones de curvatura alrededor del eje central:

$$0 \leq \phi < 2\pi \quad (31)$$

De esta forma, el espacio de trabajo ideal del manipulador puede aproximarse como el conjunto de puntos generados al variar θ y ϕ dentro de sus límites admisibles. En la práctica, este espacio puede verse reducido por limitaciones mecánicas del prototipo, rozamientos en los tendones, holguras en el sistema de actuación y deformaciones no ideales de la columna central flexible.

En este proyecto, la definición del espacio de trabajo se utiliza como una herramienta para comprobar si las posiciones objetivo empleadas en simulación pertenecen a una región alcanzable

por el manipulador. Por ejemplo, en la validación de la cinemática inversa se selecciona una posición dentro del espacio de trabajo teórico, evitando configuraciones que exigirían una curvatura superior a la admisible o una variación de longitud de cable fuera del recorrido disponible.

El espacio de trabajo también permite interpretar las limitaciones del diseño. Un manipulador más largo aumenta el alcance máximo, pero puede reducir la precisión y aumentar la deformación debida a su propio peso. Por otro lado, una mayor curvatura máxima permite alcanzar puntos más alejados lateralmente del eje central, aunque exige mayores tensiones en los tendones y puede comprometer la estabilidad estructural. Por tanto, el diseño final debe equilibrar alcance, curvatura, rigidez y capacidad de actuación.

6. Simulación

6.1. Entorno de simulación

La validación del modelo cinemático se implementa en CoppeliaSim mediante una escena específica del manipulador continuo. El objetivo de esta simulación no es reproducir con detalle todos los efectos físicos del robot real, sino comprobar que las ecuaciones de cinemática directa e inversa descritas en el apartado anterior generan configuraciones coherentes con el movimiento esperado del manipulador.

El robot se modela como una cadena de cuatro segmentos consecutivos. Cada segmento dispone de dos articulaciones de flexión ortogonales, lo que permite aproximar el comportamiento continuo del manipulador mediante una estructura discreta. Esta discretización facilita la implementación en CoppeliaSim y permite comparar directamente la posición obtenida en la simulación con la posición calculada mediante el modelo cinemático.

La escena contiene dos referencias principales. La primera, denominada 00, se sitúa en la base del manipulador y se utiliza como sistema de referencia para las medidas. La segunda, denominada 04, se sitúa en el extremo final del manipulador y permite registrar la posición real alcanzada por el efector final. En las pruebas de cinemática inversa se añade además un *dummy* en la posición objetivo, con el fin de visualizar la referencia que debe alcanzar el manipulador.

Toda la lógica de control y adquisición de datos se implementa mediante scripts Lua asociados a la escena. Durante la simulación se registran los datos relevantes en archivos CSV, que posteriormente son procesados mediante scripts en Python para generar las gráficas de validación.

6.2. Implementación del algoritmo cinemático

La implementación parte directamente del modelo PCC descrito en el apartado anterior. Por tanto, no se vuelve a desarrollar la formulación completa, sino que se aplica a la geometría concreta del manipulador simulado.

Implementación de la cinemática directa

En la prueba de cinemática directa se imponen valores de flexión conocidos sobre las articulaciones del manipulador y se calcula la posición teórica del efector final mediante la composición de las transformaciones homogéneas de cada segmento.

Cada segmento de la simulación tiene una longitud de 0,076847 m. Para cada uno de ellos se obtiene una matriz de transformación a partir de sus dos componentes de flexión. Debido a la convención de ejes utilizada en la escena de CoppeliaSim, las componentes aplicadas a las articulaciones no coinciden directamente con las componentes geométricas del modelo, por lo que se realiza una conversión interna entre los ejes de simulación y los ejes del modelo PCC.

Una vez calculada la matriz correspondiente a cada segmento, la pose final del manipulador se obtiene mediante la multiplicación sucesiva de las cuatro transformaciones. La posición teórica del efector final se extrae de la componente de traslación de la matriz resultante. Esta posición se compara posteriormente con la posición real medida en CoppeliaSim a partir de la referencia 04.

Implementación de la cinemática inversa

En la prueba de cinemática inversa se define una posición cartesiana objetivo para el efector final. A partir de dicha posición se calculan los parámetros de curvatura del modelo PCC, siguiendo la formulación descrita previamente. Estos parámetros se convierten después en las componentes de flexión que deben aplicarse sobre las articulaciones de la simulación.

Como el manipulador está discretizado en cuatro segmentos, la flexión total calculada por la cinemática inversa se reparte uniformemente entre todos ellos. De esta forma, cada segmento recibe una fracción de la curvatura total, manteniendo una aproximación coherente con la hipótesis de curvatura constante.

Además de las componentes de flexión, el algoritmo calcula la variación de longitud de los tres cables. Tal como se explicó en el apartado anterior, estos valores no representan directamente la longitud absoluta del cable, sino su acortamiento o alargamiento relativo respecto a la configuración neutra. Por ello, en la implementación se calcula también la longitud efectiva de cada cable a partir de la longitud de referencia definida para la configuración inicial.

En la prueba realizada se emplea una longitud de referencia de cable de 0,25 m. De esta forma, las variaciones de longitud calculadas por el modelo se convierten en longitudes reales que pueden analizarse durante la simulación.

6.3. Validación del modelo

La validación se realiza mediante dos pruebas independientes. La primera evalúa la cinemática directa, comparando la posición teórica calculada por el modelo con la posición real medida en CoppeliaSim. La segunda evalúa la cinemática inversa, comprobando si la configuración calculada a partir de una posición objetivo permite que el manipulador se aproxime correctamente a dicha referencia.

En ambas pruebas, el movimiento se aplica de forma progresiva durante 8 segundos. Para evitar discontinuidades en las articulaciones, se utiliza una función de suavizado de tipo *smoothstep*. Esta función permite que el movimiento comience y termine con velocidad nula, evitando saltos bruscos en la simulación.

Durante la ejecución se registran en archivos CSV la posición del efector final, los valores teóricos calculados, el error cartesiano y las variables internas relevantes del modelo. El error se calcula como la distancia euclídea entre la posición de referencia y la posición real alcanzada por el extremo del manipulador.

6.3.1. Análisis cinemático

- Análisis de cinemática directa.

En la validación de la cinemática directa se aplica el mismo valor de flexión a los cuatro segmentos del manipulador. La configuración final utilizada corresponde a una flexión de:

$$b_x = b_y = 0,271599 \text{ rad} \quad (32)$$

Con esta configuración, la posición real final medida en CoppeliaSim es:

$$p_{real} = \begin{bmatrix} 0,141586 \\ -0,126749 \\ 0,198783 \end{bmatrix} \text{ m} \quad (33)$$

Mientras que la posición teórica calculada mediante el modelo de cinemática directa es:

$$p_{teo} = \begin{bmatrix} 0,136606 \\ -0,136606 \\ 0,199952 \end{bmatrix} \text{ m} \quad (34)$$

El error cartesiano final entre ambas posiciones es:

$$e_{final} = 0,011105 \text{ m} \quad (35)$$

equivalente a aproximadamente 11,1 mm. El error medio durante la trayectoria es:

$$e_{medio} = 0,003716 \text{ m} \quad (36)$$

Estos resultados muestran que la cinemática directa reproduce de forma coherente el movimiento observado en la simulación. La diferencia entre la posición teórica y la real puede atribuirse principalmente a la discretización del manipulador continuo en cuatro segmentos, a la convención de ejes utilizada en CoppeliaSim y a las pequeñas diferencias entre el modelo PCC ideal y la representación geométrica implementada en la escena.

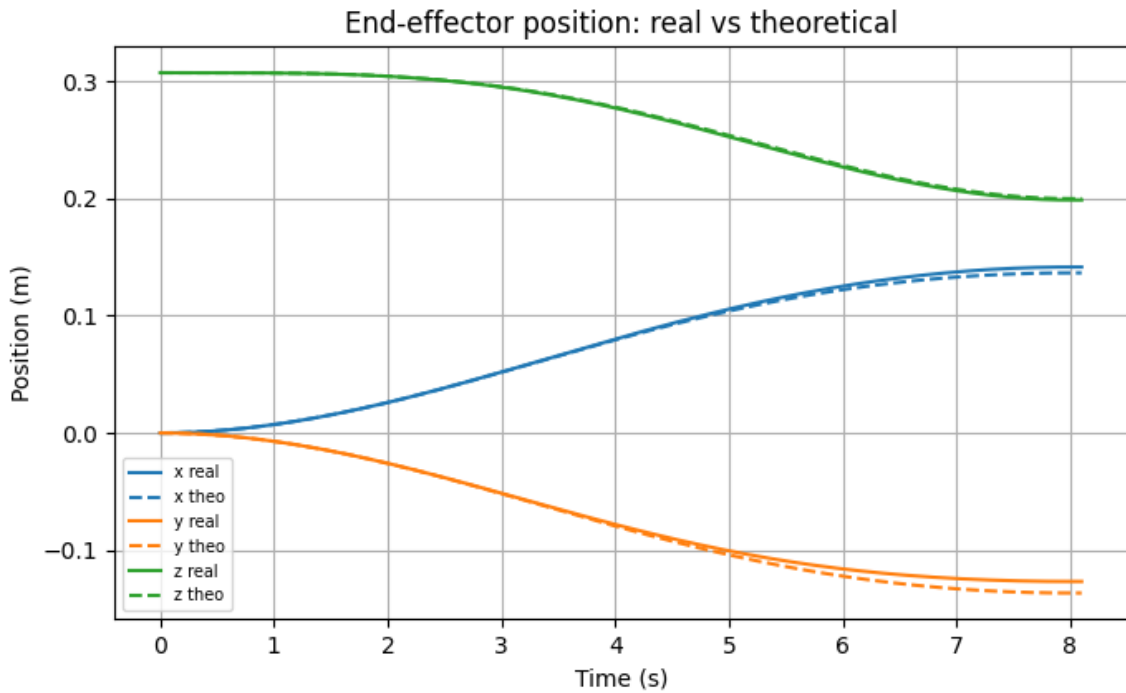


Figura 6: Validación de la cinemática directa. La figura compara la posición real y teórica del efector final, y muestra la evolución del error cartesiano.

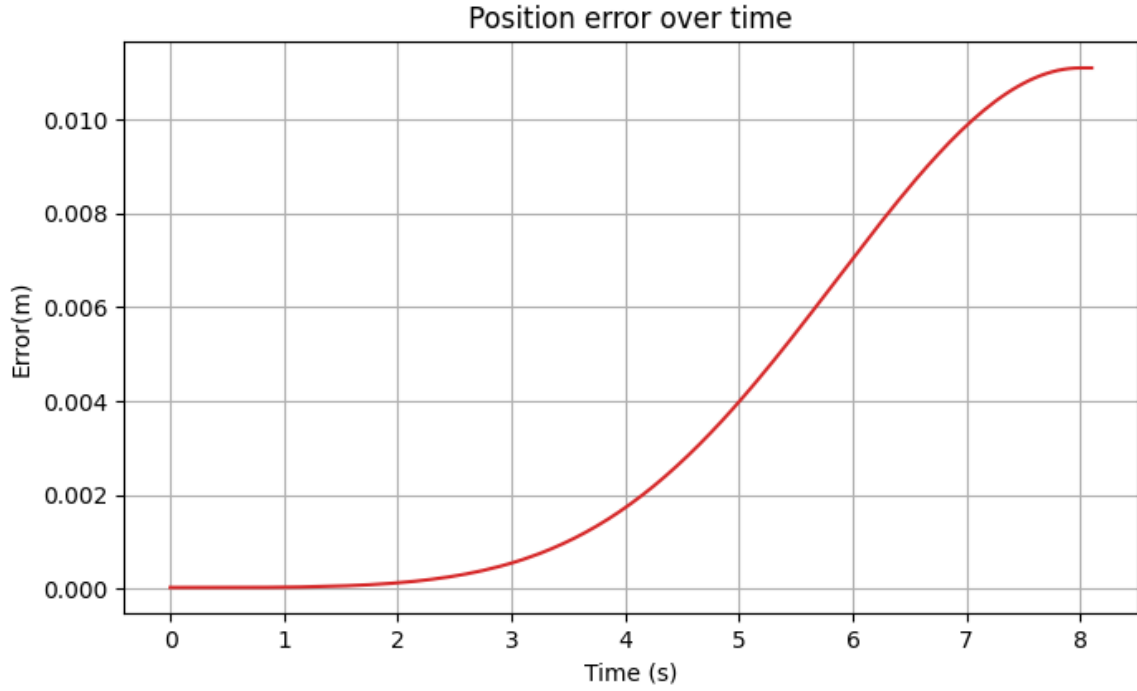


Figura 7: Validación de la cinemática directa. Muestra la evolución del error cartesiano.

La Figura 6 resume los resultados obtenidos en la prueba de cinemática directa. En la comparación de posición se observa que las componentes cartesianas calculadas mediante el modelo siguen una evolución similar a las medidas en CoppeliaSim. La Figura 7 resume el aumento del error cartesiano a lo largo del tiempo. La diferencia entre ambas trayectorias aumenta progresivamente a medida que el manipulador se curva, alcanzando un valor final de aproximadamente 11,1 mm. Este comportamiento es coherente con la aproximación discreta utilizada para representar el manipulador continuo.

- Análisis de cinemática inversa.

En la validación de la cinemática inversa se utiliza como referencia una posición perteneciente al espacio de trabajo del manipulador:

$$p_d = \begin{bmatrix} 0,136600 \\ -0,136600 \\ 0,200000 \end{bmatrix} \text{ m} \quad (37)$$

A partir de esta posición, el algoritmo calcula los parámetros de curvatura necesarios para alcanzarla. Los valores obtenidos son:

$$\theta = 1,536116 \text{ rad} \quad (38)$$

$$\phi = -0,785398 \text{ rad} \quad (39)$$

Estos parámetros se convierten en las componentes de flexión de las articulaciones y se reparten uniformemente entre los cuatro segmentos del manipulador. La posición final alcanzada por el efector final en CoppeliaSim es:

$$p_{real} = \begin{bmatrix} 0,141568 \\ -0,126739 \\ 0,198817 \end{bmatrix} \text{ m} \quad (40)$$

El error cartesiano final entre la posición objetivo y la posición alcanzada es:

$$e_{final} = 0,011105 \text{ m} \quad (41)$$

Este error es del mismo orden que el obtenido en la prueba de cinemática directa. Por tanto, la cinemática inversa calcula una configuración compatible con el modelo directo, y las diferencias restantes se deben principalmente a la aproximación discreta del robot continuo en la simulación.

Para esta misma configuración, el algoritmo calcula las siguientes variaciones de longitud de los cables:

$$\Delta l_1 = -0,021724 \text{ m} \quad (42)$$

$$\Delta l_2 = 0,029675 \text{ m} \quad (43)$$

$$\Delta l_3 = -0,007952 \text{ m} \quad (44)$$

Al aplicar la relación entre variación y longitud efectiva, considerando una longitud de referencia de 0,25 m, se obtienen las siguientes longitudes finales:

$$l_1 = 0,271724 \text{ m} \quad (45)$$

$$l_2 = 0,220325 \text{ m} \quad (46)$$

$$l_3 = 0,257952 \text{ m} \quad (47)$$

Estos valores indican que, para la configuración analizada, el segundo cable se acorta respecto a la configuración neutra, mientras que el primero y el tercero se alargan. Este comportamiento es coherente con el plano de curvatura calculado por la cinemática inversa.

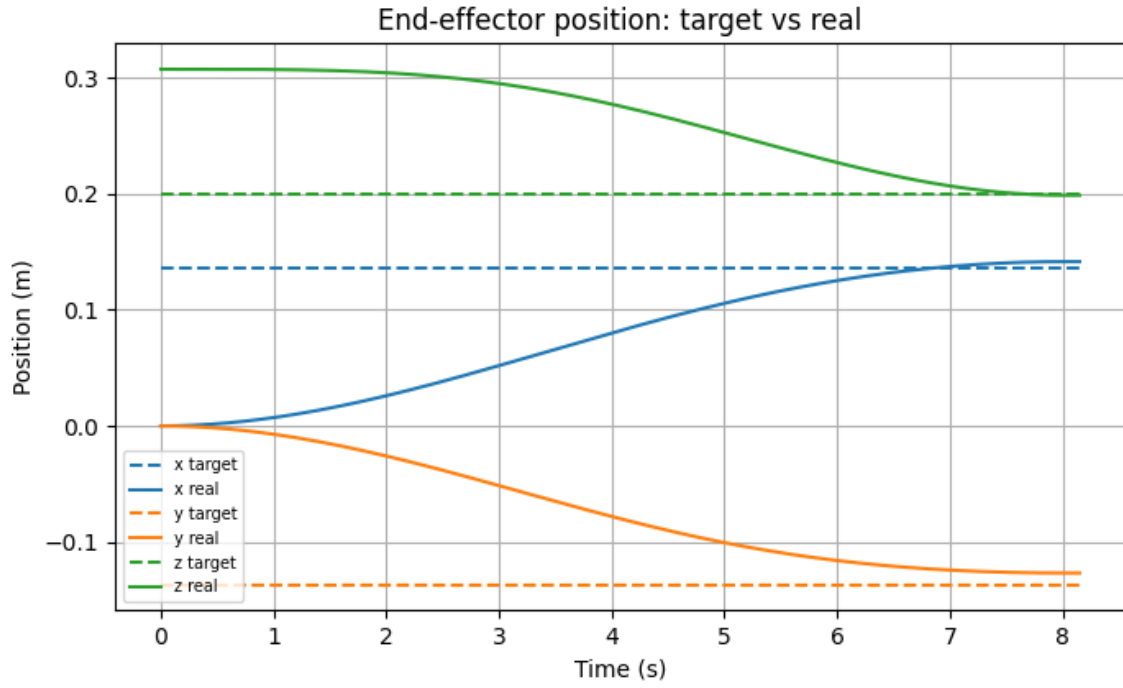


Figura 8: Validación de la cinemática inversa. La figura muestra la comparación entre la posición objetivo.

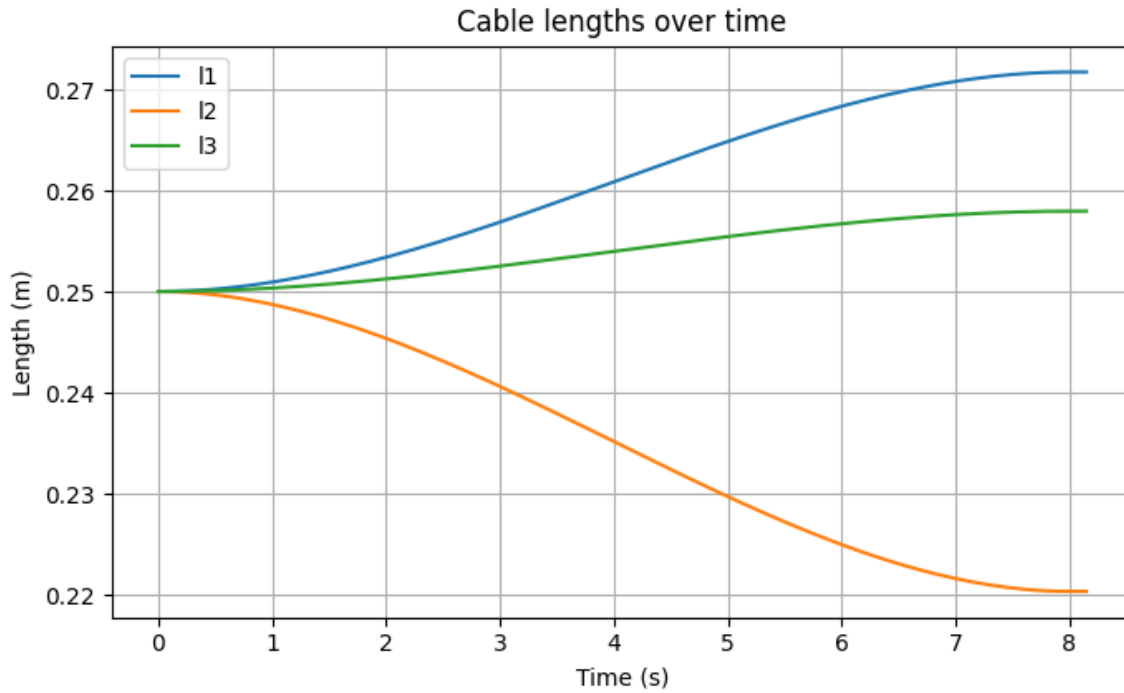


Figura 9: Validación de la cinemática inversa. La figura muestra las longitudes efectivas de los cables.

La Figura 8 resume los resultados obtenidos en la prueba de cinemática inversa. En la gráfica se observa cómo el efector final se aproxima progresivamente a la referencia definida. La evolución del error confirma la convergencia hacia el entorno del punto objetivo. Además, la Figura 9 muestra las variaciones en las longitudes de cable en el tiempo. Esta figura muestra que las longitudes varían de forma progresiva durante la rampa de movimiento y alcanzan valores coherentes con el plano de curvatura calculado.

En conjunto, los resultados obtenidos permiten validar la coherencia del modelo cinemático implementado. La cinemática directa reproduce adecuadamente la posición del efector final, mientras que la cinemática inversa calcula una configuración de flexión capaz de aproximar el extremo del manipulador a la posición objetivo. Los errores observados son reducidos y se mantienen dentro de un rango compatible con la aproximación discreta utilizada en el entorno de simulación.

7. Diseño detallado: hardware

7.1. Diseño mecánico en CAD

El diseño mecánico del manipulador se ha desarrollado con el objetivo de obtener una estructura modular, escalable, ligera y fabricable mediante impresión 3D. El modelo CAD nos permite diseñar detalladamente el diseño conceptual fijado en los apartados anteriores y llevar a la

realidad el modelo aproximado realizado en simulación.

El manipulador se compone principalmente de varios discos o segmentos separadores, tres tendones distribuidos de manera equidistante a la columna central, una base donde fijar los actuadores y microcontroladores. Debido a la configuración específica, este manipulador precisa también de una polea y un soporte para distribuir las fuerzas del sistema.

7.1.1. Diseño de segmentos

Los segmentos se han diseñado como discos circulares cuya función principal es la guía de los tendones y mantener una separación entre tendones. Cada disco contiene un orificio central para el paso de la columna central y también tiene tres orificios para el paso de los tendones. Estos tendones se distribuyen entre sí para que cada tendón produzca una flexión en una dirección determinada.

La distancia radial entre los orificios de los tendones y el centro del segmento es menor para respetar la relación entre la capacidad de flexión y el tamaño del manipulador. Una mayor distancia implica el aumento del momento generado por cada tendón y también aumenta el diámetro exterior del manipulador. Por eso, el diámetro es reducido sin comprometer la capacidad de curvatura.

7.1.2. Base del manipulador

La base del manipulador se diseña para realizar una fijación entre los actuadores y los tendones. Tiene los orificios para los tendones como en los segmentos para que los tendones salgan alineados hacia los respectivos actuadores, evitando fricciones. También tiene los orificios para fijar el sistema de actuación.

Para respetar los requisitos de diseño, todo el sistema de actuación se encuentra en la parte superior de la base. Los únicos orificios totales pasantes son los de la columna central y los orificios de los tendones, de forma que queda aislado el sistema de actuación del manipulador.

7.1.3. Polea de tendones

La polea de tendones se ha diseñado como el elemento encargado de transformar el giro del actuador en una variación lineal de la longitud del tendón. Al girar el motor en un sentido, el tendón se enrolla sobre la polea y se produce su acortamiento efectivo; al girar en el sentido contrario, el tendón se libera y permite que la elasticidad del manipulador recupere parcial o totalmente su posición neutra.

La polea se fija solidariamente al eje del actuador, de forma que no exista movimiento relativo entre ambos elementos. Esta unión debe transmitir el par del motor al tendón sin holguras

apreciables, ya que cualquier deslizamiento modificaría la longitud real recogida y afectaría a la precisión del movimiento del manipulador.

7.1.4. Soporte

El soporte del rodamiento ha sido diseñado como una pieza estructural encargada de alojar el rodamiento sobre el que gira el eje de la polea. Su función principal es mantener la polea correctamente posicionada y alineada, permitiendo su giro libre y reduciendo la fricción durante el movimiento del cable.

Además de servir como alojamiento para el rodamiento, el soporte permite transmitir a la estructura las fuerzas generadas por la tensión del cable. De esta forma, la pieza contribuye a contrarrestar los esfuerzos producidos durante el accionamiento del sistema, evitando desplazamientos indeseados de la polea y mejorando la estabilidad mecánica del conjunto.

7.1.5. Criterios de diseño

Durante el diseño CAD se han considerado los siguientes criterios:

- Modularidad, para facilitar el montaje, sustitución y modificación de los segmentos.
- Bajo peso, con el fin de reducir la carga soportada por la columna central flexible.
- Fabricabilidad mediante impresión 3D, evitando geometrías complejas o imposibles de imprimir.
- Guiado correcto de los tendones, reduciendo rozamientos y desviaciones no deseadas.
- Simetría geométrica, para obtener una respuesta de flexión lo más uniforme posible.
- Compatibilidad con el modelo cinemático, manteniendo una distribución regular de los tendones.

7.2. Sistema de actuación

7.2.1. Selección y dimensionamiento de actuadores

Para el movimiento de las articulaciones del sistema, se requiere un actuador que combine un par motor elevado, precisión en el control de posición y facilidad de integración. Tras analizar los requerimientos de carga, se ha optado por el servo inteligente Lewansoul LX-16A.

- Especificaciones técnicas del motor.

El modelo LX-16A es un servo de bus serie que permite una retroalimentación en tiempo real de parámetros como la posición, temperatura y voltaje. En la Tabla 1 se resumen sus características principales:

Cuadro 1: Especificaciones técnicas del servo Lewansoul LX-16A.

Parámetro	Valor
Modelo	LX-16A
Par máximo (Stall Torque)	17 kg · cm ($\approx 1,6677 \text{ N} \cdot \text{m}$)
Voltaje de operación	6,0 V – 8,4 V
Rango de rotación	0° a 240°
Comunicación	Serial UART (Bus serie)

- Análisis mecánico y validación de la carga.

Para validar que el motor seleccionado es capaz de soportar el peso del brazo y la carga útil, se realiza un análisis estático considerando el peor caso (brazo en extensión máxima horizontal). Se definen los siguientes parámetros de diseño:

- **Longitud del brazo (L):** 25 cm (0,25 m).
- **Factor de seguridad (f_s):** 1,25 (para compensar rozamientos e inercias).
- **Par máximo disponible (τ_M):** 17 kg · cm.

En primer lugar, se calcula la fuerza máxima ($F_{\text{max_servo}}$) que el actuador puede ejercer sobre un radio de polea específico de 1,25 cm:

$$F_{\text{max_servo}} = \frac{\tau_M}{r} = \frac{17 \text{ kg} \cdot \text{cm}}{1,25 \text{ cm}} = 13,6 \text{ kg} \approx 133,28 \text{ N} \quad (48)$$

A continuación, determinamos la masa máxima (m) que el servo puede mantener en equilibrio estático en el extremo del brazo. Igualando el par motor al par de carga ($\tau_L = \tau_M$) y aplicando el factor de seguridad, se obtiene:

$$\tau_L = m \cdot g \cdot L \cdot f_s \quad (49)$$

Despejando la masa m :

$$m = \frac{\tau_M}{g \cdot L \cdot f_s} = \frac{1,6677 \text{ N} \cdot \text{m}}{9,81 \text{ m/s}^2 \cdot 0,25 \text{ m} \cdot 1,25} \quad (50)$$

$$m \approx 0,544 \text{ kg} = 544 \text{ g} \quad (51)$$

El cálculo indica que el actuador es capaz de manipular una masa de hasta 544 g a una distancia de 25 cm manteniendo un margen de seguridad del 25 %. Dado que el peso proyectado de la estructura y la carga es inferior a este límite, se ratifica la elección del modelo LX-16A.

7.3. Electrónica y control

7.3.1. Microcontrolador

7.3.2. Esquema eléctrico

7.4. Fabricación

8. Diseño detallado: arquitectura software

8.1. Integración en ROS 2

Esta arquitectura software del manipulador se ha desarrollado sobre ROS2, compuesta por tres paquetes fundamentales que dividen la descripción del robot, la cinemática, el lanzamiento y visualización del sistema. Gracias a ello, obtenemos una arquitectura modular que nos permite desacoplar las diferentes partes facilitando el mantenimiento del código y la escalabilidad del mismo.

- *cable_robot_bringup* : contiene el lanzamiento completo que relaciona la visualización del robot y la cinemática obteniendo una actuación simulada.
- *cable_robot_description* : contiene la descripción del robot en formato *URDF/XACRO*, los recursos gráficos o mallas, la configuración de *RViz* y un lanzamiento básico para la visualización.
- *cable_robot_hardware*: contiene el plugin de interfaz hardware para *ros2_control*, responsable de la comunicación serie con el Arduino y de la integración del estado de posición de los cables.
- *cable_robot_kinematics* : contiene los nodos encargados del cálculo de la cinemática y los nodos encargados de relacionar los parámetros cartesianos y la variación en la longitud de los cables.

8.1.1. Creación del paquete URDF/Xacro del robot

Con el fin de de obtener una representación digital del manipulador en el ecosistema de ROS2 se desarrollo el paquete *cable_robot_description* que contiene la descripción geométrica y cinemática del manipulador, además contiene una simple visualización en *RViz* y las mallas que constituyen la geometría del manipulador y obtenidas del diseño mecánico en CAD.

Se optó por emplear *XACRO* en lugar de un *URDF* estático, ya que permite parametrizar el modelo y reutilizar estructuras repetitivas mediante macros. Esta decisión resulta especialmente útil en un robot segmentado y accionado por cables, donde existen elementos que se repiten varias veces, como los segmentos del cuerpo y las secciones visuales de los cables. Gracias a ello, el modelo es más legible, mantenible y escalable ante futuros cambios de diseño.

El archivo principal *cable_robot.urdf.xacro* define en primer lugar una serie de parámetros como la ruta de las mallas, radio, posición y longitud de los cables, espacio entre cada sección, etc. Estos parámetros definidos nos permiten modificar el modelo sin necesidad de reescribir los bloques del fichero. Después se definen las macros:

- *segment_stage* : encapsula la creación de los enlaces intermedios, las articulaciones de flexión y la colocación del segmento visual correspondiente.
- *cable_link* : define la parte activa de cada cable, creando un enlace de anclaje unido a la base y una articulación prismática que representa su desplazamiento lineal.
- *cable_section* : se utiliza para representar visualmente los distintos tramos del cable a lo largo del robot. Cada llamada a esta macro genera un enlace con geometría cilíndrica unido mediante una articulación fija al segmento correspondiente. Así, aunque estas secciones no añaden movilidad al modelo, permiten mostrar en RViz la trayectoria aproximada de cada cable y su disposición espacial respecto a la estructura segmentada del robot

A nivel estructural, el robot se organiza a partir de un enlace base (*base_link*), unido al sistema de referencia global mediante una articulación fija. Sobre la base se constituyen cuatro segmentos representados mediante enlaces sucesivos. Cada sección se implementa con dos articulaciones de revolución ortogonales, alrededor del eje X y alrededor del eje Y. Esta implementación permite la flexión en dos planos requerida para este manipulador.

Finalmente obtenemos un paquete que nos permitió la visualización y validación del robot para la posterior integración con el resto de paquetes de la arquitectura.

8.1.2. Integración de *ros2_control*

Para poder controlar los servos LX-16A de forma coherente dentro del ecosistema ROS 2, se integró *ros2_control*, el framework estándar para control de hardware en tiempo real. Esta integración aporta tres ventajas fundamentales: separación limpia entre lógica de control y hardware, ciclo de control determinista, y compatibilidad con los controladores estándar de *ros2_controllers*.

La arquitectura de *ros2_control* en este sistema se estructura en tres capas:

1. **Controller Manager**: proceso central que carga el plugin hardware, gestiona el ciclo de actualización a 10 Hz (configurado en *controllers.yaml*) y orquesta el ciclo *read* → *update* → *write* en cada iteración.
2. **Controladores**: se activan dos controladores estándar:
 - *JointStateBroadcaster*: lee las interfaces de estado de posición del plugin hardware y las publica periódicamente en el tópico */joint_states*, proporcionando retroalimentación del estado estimado de los cables al resto del sistema.

- *ForwardCommandController* (*cable_velocity_controller*): recibe un vector de velocidades mediante el tópic `/cable_velocity_controller/commands` (tipo `Float64MultiArray`) y los escribe directamente en las interfaces de comando de velocidad del plugin hardware.

3. Plugin hardware (*CableRobotHardware*):

implementa la interfaz `hardware_interface::SystemInterface` y constituye la frontera entre el software de control y el hardware físico. Se detalla en la siguiente sección.

8.1.3. Plugin hardware: *CableRobotHardware*

El plugin *CableRobotHardware*, implementado en C++ en el paquete *cable_robot_hardware*, gestiona la comunicación serie con el Arduino y mantiene el estado estimado de posición de los tres cables. Al seguir el modelo de ciclo de vida de *ros2_control*, expone cuatro transiciones de estado:

- `on_init`: lee los parámetros del URDF (`port`, `baud`, `cable_speed`) e inicializa los vectores de posición y velocidad a cero.
- `on_configure`: abre el puerto serie con los flags `O_RDWR | O_NOCTTY | O_NONBLOCK`, configura la velocidad de comunicación (9600 baudios) y aplica los parámetros de la terminal serie. El flag `O_NONBLOCK` es imprescindible para que la llamada `write()` no bloquee el hilo de tiempo real del *controller_manager*, cuyo ciclo de 100 ms no admite latencias adicionales.
- `on_activate` / `on_deactivate`: envían el comando de velocidad cero ("`v:0,0,0\n`") al Arduino para garantizar que los servos queden parados al iniciar o detener el sistema.

En cada iteración del ciclo de control se ejecutan dos métodos:

- `read()`: integra la posición de cada cable a partir de la velocidad comandada en la iteración anterior, según la expresión $p_i += v_i \cdot k_v \cdot \Delta t$, donde $k_v = 0,026$ es el factor de conversión entre la velocidad normalizada y el desplazamiento real en metros. Esta integración proporciona una estimación de la longitud actual de cada cable publicada posteriormente por el *JointStateBroadcaster*.
- `write()`: construye el mensaje de protocolo "`v:spd0,spd1,spd2\n`" con las velocidades escaladas a enteros (multiplicadas por 1000) y lo envía al Arduino por el puerto serie. Cabe destacar que los índices de los cables 1 y 2 están intercambiados en el código ($spd0 \leftarrow v_1$, $spd1 \leftarrow v_0$) para compensar el cableado físico inverso del prototipo.

8.1.4. Definición de nodos

La arquitectura en ejecución se compone de los siguientes nodos:

- *robot_state_publisher* : publica las transformaciones del robot a partir del modelo *URDF* generado desde *XACRO* y del t3pico */joint_states*.
- *ik_bridge* : recibe una posici3n objetivo del t3pico */desired_pose* y calcula las longitudes equivalentes de los cables usando la funci3n *get_cable_pull* de la librería *pcc_kinematics* y los parámetros de curvatura obtenidos de la funci3n *per_segment_bending*. Las longitudes obtenidas se publican en el t3pico */cable_joint_commands*. Adem3s, calcula los ángulos de flexi3n de cada segmento obtenidos tambi3n con al funci3n *per_segment_bending* de la librería *pcc_kinematics* y publica los ángulos de flexi3n en el t3pico */bend_joint_commands*.
- *fk_bridge*: implementa la cinemática directa real basada en el modelo PCC. Suscribe al t3pico */joint_states* para obtener las posiciones actuales integradas de los cables, calcula la posici3n cartesiana del extremo del manipulador mediante la funci3n *fwd_kinematics* de la librería *pcc_kinematics*, y publica el resultado en */estimated_pose*. La derivaci3n matemática se describe en detalle en la secci3n 8.1.5.
- *joint_state_combiner* : este nodo recoge toda la informaci3n de los t3picos */bend_joint_commands*, */cable_joint_states*, */manual_cable_joint_states*, y genera un único mensaje con el estado completo del robot y lo publica en */joint_states* utilizado en *robot_state_publisher*.
- *cable_bend_bridge* : este nodo actúa de puente ya que recibe las variaciones en las longitudes de los cables del t3pico */cable_joint_states* y calcula el ángulo de flexi3n de cada segmento. La salida se publica en el t3pico */manual_cable_joint_states*.
- *hardware_controller*: controlador de posici3n en espacio de cables implementado en Python. Suscribe a */manual_cable_joint_commands* para obtener la posici3n deseada de cada cable y a */joint_states* para obtener la posici3n actual estimada. Su l3gica de control se describe en la secci3n 8.1.6.
- *cable_slider_gui*: interfaz gr3fica de usuario descrita en la secci3n 8.1.10.

De forma adicional, se ha desarrollado un nodo de control manual denominado *cable_slider_gui*, implementado en Python con una interfaz basada en deslizadores. Este nodo publica en */manual_cable_joint_commands* y permite modificar directamente la elongaci3n o retracci3n relativa de cada cable. Permitiendo realizar pruebas de validaci3n y depuraci3n del comportamiento cinemático.

8.1.5. Cinemática directa: modelo PCC

La cinemática directa calcula la posici3n cartesiana del extremo del robot a partir de las variaciones de longitud de los cables Δl_i , obtenidas por integraci3n en el plugin hardware.

El modelo de cable en curvatura constante por tramos establece que la variación de longitud del cable i respecto a la posición de reposo es:

$$\Delta l_i = d_c \cdot \theta \cdot \sin(\varphi + \varphi_{c_i})$$

donde $d_c = 0,02$ m es el radio de separación de los cables al eje neutro, θ es el ángulo de curvatura total del robot, φ es el ángulo del plano de flexión, y φ_{c_i} es la posición angular del cable i ($\varphi_{c_1} = 0$, $\varphi_{c_2} = 2\pi/3$, $\varphi_{c_3} = 4\pi/3$).

Expandiendo las tres ecuaciones con los ángulos de los cables y operando algebraicamente se obtiene el sistema:

$$d_c \cdot \theta \cdot \sin \varphi = \Delta l_1 \quad d_c \cdot \theta \cdot \cos \varphi = \frac{\Delta l_2 - \Delta l_3}{\sqrt{3}}$$

De donde se despeja directamente el ángulo de curvatura total y el ángulo del plano de flexión:

$$\theta = \frac{1}{d_c} \sqrt{\Delta l_1^2 + \frac{(\Delta l_2 - \Delta l_3)^2}{3}} \quad \varphi = \arctan 2 \left(\Delta l_1, \frac{\Delta l_2 - \Delta l_3}{\sqrt{3}} \right)$$

Con θ y φ recuperados, la posición del extremo se obtiene aplicando la geometría de arco circular del modelo PCC con longitud de arco total $L = n_{seg} \cdot d_{seg} = 4 \times 0,05675 = 0,227$ m:

$$\mathbf{p}_{tip} = \begin{cases} (0, 0, 0) & \text{si } \theta < \varepsilon \\ \left(\frac{L}{\theta} (1 - \cos \theta) \cos \varphi, \frac{L}{\theta} (1 - \cos \theta) \sin \varphi, \frac{L}{\theta} \sin \theta - L \right) & \text{en caso contrario} \end{cases}$$

El término $-L$ en la componente z sitúa el origen del sistema de referencia en la posición del extremo con el robot extendido verticalmente, de manera que la posición de reposo corresponde a $(0, 0, 0)$ y las coordenadas expresan el desplazamiento del extremo respecto a dicha posición.

8.1.6. Controlador de posición en espacio de cables

El nodo *hardware_controller* implementa un controlador proporcional con banda muerta en el espacio de longitudes de cable. En cada ciclo de 100 ms calcula, para cada uno de los tres cables, el error entre la longitud deseada y la estimada:

$$e_i = l_{d,i} - l_{a,i}$$

Si $|e_i| < 2$ mm el cable se considera en reposo y se envía velocidad cero para evitar oscilación

en estado estacionario. En caso contrario, la velocidad de salida se calcula como:

$$v_i = \text{clip}(K_P \cdot e_i, -V_{max}, V_{max})$$

Adicionalmente, se impone una velocidad mínima de actuación para vencer el rozamiento estático y el peso de los cables:

$$|v_i| \geq \begin{cases} V_{min}^{tens} = 0,15 & \text{si el cable se tensa } (v_i > 0) \\ V_{min}^{suelt} = 0,185 & \text{si el cable se suelta } (v_i < 0) \end{cases}$$

La asimetría entre las dos velocidades mínimas responde al comportamiento físico del sistema: soltar un cable requiere vencer tanto el rozamiento como la tensión residual ejercida por el peso del extremo del robot. El vector de velocidades se publica en `/cable_velocity_controller/commands`, donde el *ForwardCommandController* lo escribe directamente en las interfaces de comando del plugin hardware.

8.1.7. Grafo de la arquitectura

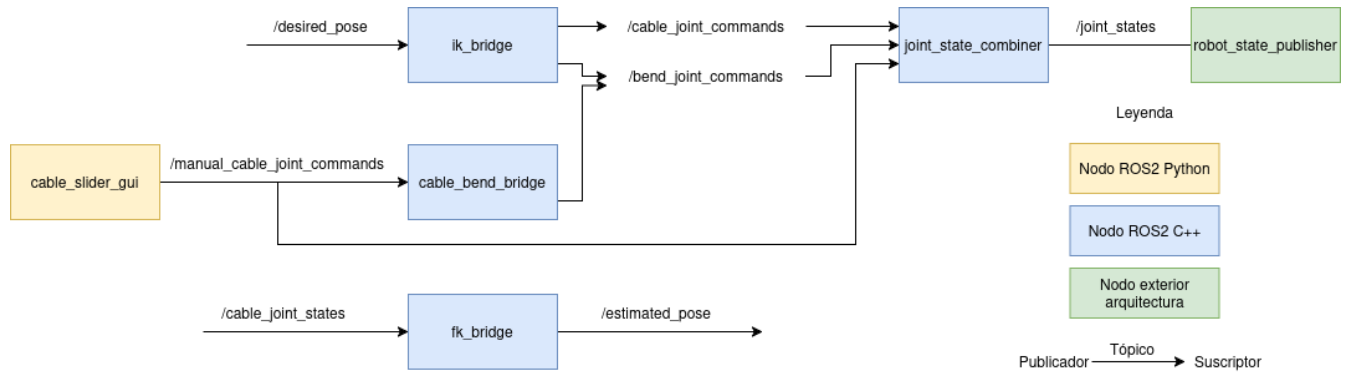


Figura 10: Grafo de la arquitectura software en ROS2

8.1.8. Modos de operación y comando del robot

El sistema admite dos formas de enviar comandos al manipulador, que responden a perfiles de uso distintos: una orientada a la experimentación técnica y otra pensada para un control más intuitivo.

- Comando mediante tópicos ROS 2

Dado que toda la arquitectura se articula sobre ROS 2 y *ros2_control*, es posible comandar el robot directamente desde la línea de órdenes o desde cualquier nodo externo, sin necesidad de ninguna interfaz gráfica adicional. Esto resulta especialmente útil durante fases de depuración, validación cinemática o integración con otros sistemas.

Existen dos puntos de entrada según el nivel de abstracción deseado:

- **Nivel cartesiano (a través del IK):** publicando un mensaje `PoseStamped` en el tópico `/desired_pose`, el nodo `ik_bridge` calcula automáticamente las longitudes de cable equivalentes. Este es el nivel de abstracción más alto y el más natural para especificar poses del extremo:

```
ros2 topic pub --once /desired_pose \
geometry_msgs/msg/PoseStamped \
"{pose: {position: {x: 0.1, y: 0.0, z: 0.9}}}"
```

- **Nivel de velocidad de cable (acceso directo al controlador):** publicando un vector `Float64MultiArray` en `/cable_velocity_controller/commands` se envían velocidades directamente al *ForwardCommandController*, sin pasar por la cinemática. Útil para pruebas de actuación individuales:

```
ros2 topic pub /cable_velocity_controller/commands \
std_msgs/msg/Float64MultiArray \
"{data: [-0.2, 0.2, 0.0]}"
```

Esta interfaz basada en tópicos permite también integrar el manipulador con herramientas gráficas estándar de ROS 2 como *rqt_publisher*, que ofrece una interfaz visual genérica para publicar en cualquier tópico sin escribir código.

- Comando mediante interfaz gráfica.

Para un control más intuitivo y continuo del robot se desarrolló el nodo *cable_slider_gui*, descrito en la sección anterior. A diferencia del comando por tópicos, la GUI integra en una única ventana tanto el envío de comandos como la monitorización del estado estimado, lo que la hace más adecuada para sesiones de experimentación prolongadas donde se necesita observar la respuesta del robot en tiempo real.

La GUI soporta los mismos dos niveles de abstracción que el comando por tópicos: control directo de la longitud de cada cable mediante deslizadores, y control por posición cartesiana mediante la sección IK. Adicionalmente, muestra de forma continua la posición estimada del extremo calculada por la cinemática directa y el error cartesiano respecto al objetivo, información que no está disponible de forma inmediata al usar `ros2 topic pub`.

Ambas vías de comando son completamente compatibles entre sí: es posible, por ejemplo, enviar una posición objetivo desde la línea de órdenes y observar la respuesta en la GUI, o usar los deslizadores para ajuste fino tras un posicionamiento grueso por cinemática inversa.

8.1.9. Configuración de RViz para visualización

La visualización del robot en RViz se configura a través del fichero *view_robot.rviz* incluido en el paquete *cable_robot_description*. La escena muestra el modelo 3D del manipulador con las mallas del diseño CAD, cuya pose se actualiza en tiempo real a partir de las transformaciones TF publicadas por *robot_state_publisher*. Esto permite observar visualmente la deformación del robot al actuar sobre los cables, tanto en el modo de control manual como en el modo de cinemática inversa.

8.1.10. Interfaz gráfica de usuario

El nodo *cable_slider_gui*, implementado en Python con la librería Tkinter, proporciona dos modos de operación accesibles desde una única ventana:

- **Control manual por cables:** tres deslizadores permiten fijar directamente la longitud deseada de cada cable en el rango $[0, 0,15]$ m. Los valores se publican a 10Hz en `/manual_cable_joint_commands`. El nodo suscribe además a `/cable_joint_commands` para actualizar automáticamente la posición de los deslizadores cuando se envía un comando de cinemática inversa, manteniendo coherencia visual entre ambos modos.
- **Control por posición cartesiana (IK):** campos de entrada permiten especificar una posición objetivo (x, y, z) en metros. Al pulsar el botón *Ir a posición*, el nodo publica el objetivo en `/desired_pose`, que el nodo *ik_bridge* transforma en longitudes de cable. La interfaz muestra en tiempo real la posición estimada por la cinemática directa recibida en `/estimated_pose` y el error cartesiano en milímetros.

8.2. Firmware: Arduino

8.2.1. Control a bajo nivel

El firmware del Arduino actúa como capa de conversión entre los comandos de velocidad procedentes del PC y el bus de comunicación propietario de los servos LX-16A. El Arduino Uno emplea la UART hardware (pines 0 y 1) para comunicarse con el PC a 9600 baudios y una UART software (*SoftwareSerial*, pines 10 y 11) para comunicarse con los servos a 115200 baudios mediante el protocolo semidúplex de Lewansoul.

El firmware reconoce tres comandos ASCII terminados en `\n`:

- **v: s0, s1, s2:** establece la velocidad de cada servo directamente como un entero con signo (la velocidad ROS normalizada multiplicada por 1000). Un valor positivo indica tensado del cable y negativo indica aflojado.
- **c: c0, c1, c2:** recibe longitudes de cable en metros y mapea internamente cada valor a la velocidad correspondiente.

- **stop**: detiene los tres servos inmediatamente enviando velocidad cero a cada uno.

Esta interfaz minimalista mantiene la latencia de comunicación reducida y desacopla la lógica de control del hardware específico del servo, permitiendo sustituir los actuadores sin modificar el software ROS.

8.2.2. Librerías Arduino para operación autónoma sin ROS

Con el objetivo de que el manipulador pueda operar de forma completamente autónoma sin depender de un PC ni del ecosistema ROS2, se desarrollaron tres librerías en C++ para Arduino que replican en el microcontrolador toda la cadena de control: desde la comunicación de bajo nivel con los servos hasta el cálculo cinemático completo.

- Librería LX16A

La librería *LX16A* implementa el protocolo de comunicación binario semidúplex propietario de Lewansoul para los servos LX-16A. El constructor recibe una referencia a cualquier objeto *Stream* de Arduino, lo que permite usar tanto la UART hardware como una UART software sin modificar el código de la librería. La librería expone las siguientes operaciones:

- **move(id, position, time_ms)**: envía un comando de posición absoluta en el rango 0–1000 unidades con un tiempo de movimiento especificado, lo que permite que el servo interpole la trayectoria internamente.
- **setMotorSpeed(id, speed)**: activa el modo de motor de velocidad continua con valores en el rango $[-1000, 1000]$, donde el signo determina el sentido de giro. Este es el modo empleado durante el control desde ROS 2.
- **stop(id), load(id), unload(id)**: detención inmediata, activación y desactivación del par motor respectivamente.
- **readPosition(id)**: consulta la posición actual del servo mediante una transacción de petición-respuesta sobre el bus semidúplex.
- **setID(oldID, newID)**: permite reconfigurar el identificador de un servo en tiempo de ejecución.

Cada trama se construye según el formato del protocolo Lewansoul: doble byte de cabecera (0x55 0x55), identificador, longitud, comando, datos y checksum calculado como el complemento a uno de la suma de los bytes de datos.

- Librería PCCKinematics

La librería *PCCKinematics* es una cabecera con métodos estáticos que implementa el modelo cinemático PCC directamente en el Arduino, replicando en C++ embebido la misma matemática del paquete `cable_robot_kinematics` de ROS 2. Proporciona tres funciones:

- `invkinematics(x, y, z, &theta, &phi)`: calcula el ángulo de curvatura y el plano de flexión a partir de un vector de dirección del extremo, devolviendo los resultados por referencia.
- `getCablePull(theta, phi, &c1, &c2, &c3, ...)`: calcula las variaciones de longitud de los tres cables a partir de θ y φ y los ángulos de posición de cada cable.
- `fwdKinematics(c1, c2, c3, &x, &y, &z, robot_length, d_cable, n_seg)`: cinemática directa, calcula la posición cartesiana del extremo a partir de las longitudes de cable.

- Librería *TendonRobot*

La librería *TendonRobot* es la capa de más alto nivel. Encapsula una instancia de *LX16A* y de *PCCKinematics* y expone una interfaz orientada al robot completo, ocultando los detalles del protocolo serie y del cálculo cinemático:

- `moveTo(x, y, z, time_ms)`: recibe una posición cartesiana del extremo, ejecuta internamente la cinemática inversa PCC, convierte las longitudes de cable a posición de servo mediante el radio de la polea, y envía los comandos de posición a los tres servos con el tiempo de movimiento indicado.
- `moveFromCables(c1, c2, c3, time_ms)`: mueve los servos a partir de longitudes de cable directamente, sin pasar por la cinemática inversa.
- `cableToServoPos(cable_length)`: función de conversión que transforma una longitud de cable en la unidad de posición del servo, teniendo en cuenta el radio de la polea y el rango angular máximo del servo.

La combinación de estas tres librerías permite escribir sketches de Arduino que controlan el manipulador con cinemática completa en pocas líneas, sin ninguna dependencia de ROS 2 ni de un ordenador externo:

```
SoftwareSerial lx16aSerial(10, 11);
LX16A servos(lx16aSerial);
uint8_t ids[3] = {10, 4, 7};
TendonRobot robot(servos, 4, 0.02, 0.01, 0.0, 2.094, 4.189,
                  ids, 1000, 240);

void setup() { lx16aSerial.begin(115200); }

void loop() {
  robot.moveTo(0.1, 0.0, 0.9, 1500); // posición cartesiana
  delay(2000);
}
```

```
robot.moveTo(0.0, 0.0, 1.0, 1500); // posición home  
delay(2000);  
}
```

Esta capacidad de operación autónoma resulta relevante en escenarios donde se requiere un sistema embebido compacto sin la infraestructura de ROS 2, como demostraciones portátiles o futuras integraciones en plataformas con recursos limitados.

9. Resultados y pruebas experimentales

10. Conclusiones

Referencias

- [1] Diego Cerrillo, Antonio Barrientos, and Jaime Del Cerro. Kinematic modelling for hyper-redundant robots—a structured guide. *Mathematics*, 10(16), 2022.
- [2] Coppelia Documentation. <https://www.coppeliarobotics.com/>.
- [3] Reinhard M. Grassmann, Priyanka Rao, Quentin Peyron, and Jessica Burgner-Kahrs. Fas—a fully actuated segment for tendon-driven continuum robots. *Frontiers in Robotics and AI*, Volume 9 - 2022, 2022.
- [4] Timur Aygumov Islam Magomedov and Fatimat Dзамikhova. Continuum robotics: variations, applications, and limitations. *ITM Web of Conferences* 72, 2025.
- [5] Andrés Martín Barrio, Silvia Terrile, Antonio Barrientos Cruz, and Jaime del Cerro Giner. Robots hiper-redundantes: Clasificación, estado del arte y problemática. *Revista Iberoamericana de Automática e Informática Industrial*, 2018.
- [6] Priyanka Rao, Quentin Peyron, Sven Lilge, and Jessica Burgner-Kahrs. How to model tendon-driven continuum robots and benchmark modelling performance. *Frontiers in Robotics and AI*, Volume 7 - 2020, 2021.

A. Anexos